

ODT Library

RigL Gap Analysis: Every Source File

Full RigL.h and RigL.c Implementation with Line-by-Line Explanation

Covers all src/ modules: tensor, arithmetic, layer, optimizer, training_loop, loss_functions, rng, serial, userApi

Each chapter: (1) exact RigL gap in the file, (2) implementation of the gap with technical concepts

Chapter 1 -- RigL Overview: What Every File Must Contribute

CONCEPT: RigL Algorithm (Evci et al. 2020)

RigL trains sparse neural networks by dynamically updating the sparsity MASK during training. Every T gradient steps:

PRUNE: Remove the K active weights with smallest absolute value.

Rational: small $|w|$ = low contribution to output; safe to remove.

GROW: Add the K inactive weights with largest absolute gradient.

Rational: high $|\text{grad}|$ = would reduce loss most if activated.

Algorithm per layer with weights W and mask M (1=active, 0=pruned):

1. Forward: $\text{out} = (W * M) @ \text{input}$ [apply mask before use]
2. Backward: $\text{grad_W} = \text{backward}(\text{out}, \text{loss})$ [ALL weights get gradient]
3. Every T steps:
 - a. Prune: $K = \text{floor}(S * \text{nnz}(M))$ indices = K smallest $|W[M==1]|$; $M[\text{indices}] = 0$; $W[\text{indices}] = 0$
 - b. Grow: indices = K largest $|\text{grad}[M==0]|$; $M[\text{indices}] = 1$
4. SGD step: $W -= \text{lr} * \text{grad_W}$ [only active weights get nonzero update]
5. Re-mask: $W = W * M$ [zero out any drift in pruned weights]
6. Zero grad: $\text{grad_W} = 0$

Target sparsity S: typically 80-99%. For STM32F746 (320KB RAM), $S=0.9$ can reduce a Linear(256,128) weight matrix from 128KB to 12.8KB.

CONCEPT: What Each ODT Module Must Add for RigL

Tensor.h/c: Fill sparsityConfig_t; add rigLSparsityConfig_t; mask init fns

MinMax.h/c: Add findAbsMinIndexFloat, findAbsMaxIndexFloat (for prune/grow)

Matmul.h/c: Add matmulFloat32TensorsWithMask (sparse matmul)

Arithmetic.h/c: Add maskedPointWiseArithmetic (skip inactive elements)

Linear.h/c: Apply mask in linearForwardFloat before matmul

LinearApi.h/c: Pass mask tensor in linearLayerInit

Conv1d.h/c: Apply mask in conv1dKernelFloat32 inner loop

Conv1dTransposed.h/c: Apply mask in scatter forward

Optimizer.h/c: Add re-masking after SGD step

Sgd.h/c: Skip update for inactive OR re-mask after update

TrainingEpochDefault.c: Call rigLStep every T steps

CalculateGradsSequential.c: Accumulate grad for ALL weights (inactive included)

Layer.h/c: Add RIGL_SPARSE layerType or sparsity field in each config

Serialize/Deserialize: Save/load BOOL mask alongside weights

RigL.h + RigL.c: New files implementing the full RigL update loop

Chapter 2 -- Tensor.h / Tensor.c

GAP IDENTIFIED

GAP: sparsityConfig struct is EMPTY. sparsityType_t uses placeholder names (SPARSITY_TYPE_1, SPARSITY_TYPE_2) with no implementation. tensor_t carries a sparsity_t* field but nothing populates it.

MISSING: rigLSparsityConfig_t struct, mask initialization, applyMask(), rigL step counter, target sparsity, cycle tracking.

2.1 -- Existing Code (tensor.h excerpts)

/* EXISTING -- placeholders only */	
typedef enum { SPARSITY_TYPE_1, SPARSITY_TYPE_2 } sparsityType_t;	<-- no semantic meaning
typedef struct sparsityConfig { } sparsityConfig;	<-- EMPTY -- no fields
typedef struct sparsity {	
sparsityType_t type;	<-- type tag (unused in practice)
sparsityConfig *config;	<-- points to empty struct
} sparsity_t;	
typedef struct tensor {	
uint8_t *data;	
shape_t *shape;	
quantization_t *quantization;	
sparsity_t *sparsity;	<-- field exists but always NULL
} tensor_t;	

2.2 -- Gap Implementation: rigLSparsityConfig_t

The empty sparsityConfig must be extended with a concrete RigL config.

IMPLEMENTATION

Add to Tensor.h (replace empty sparsityConfig):

```
typedef enum { SPARSITY_RIGL } sparsityType_t; /* extend as needed */
```

```
typedef struct rigLSparsityConfig {
```

```
tensor_t *mask; /* BOOL tensor, same shape as weight tensor */
```

```
float targetSparsity; /* e.g. 0.90 = 90% weights pruned */
```

```
float currentSparsity; /* tracked as mask zeros / total */
```

```
size_t updateFreq; /* prune+grow every T gradient steps */
```

```
size_t numCycles; /* total RigL update cycles to perform */
```

```

size_t stepCount; /* global gradient step counter */

size_t cycleCount; /* how many prune/grow cycles done */

} rigLSparsityConfig_t;

```

/* In Tensor.h: replace the empty sparsityConfig typedef */	
typedef struct sparsityConfig {	
/* union-style: cast config pointer to the right type based on sparsityType_t */	
} sparsityConfig_t; /* kept for ABI compat -- subtype via cast */	
/* New concrete type: */	
typedef struct rigLSparsityConfig {	
tensor_t *mask;	<-- BOOL tensor: 1=active, 0=pruned
float targetSparsity;	<-- target fraction of zero weights
float currentSparsity;	<-- tracked across prune cycles
size_t updateFreq;	<-- steps between prune/grow
size_t numCycles;	<-- max prune+grow cycles (then freeze mask)
size_t stepCount;	<-- incremented every backward pass
size_t cycleCount;	<-- incremented every prune+grow
} rigLSparsityConfig_t;	
/* Add to Tensor.h: new functions */	
void initRigLSparsityConfig(	
rigLSparsityConfig_t *cfg, tensor_t *maskBuffer,	
float targetSparsity, size_t updateFreq, size_t numCycles);	
void rigLApplyMask(tensor_t *param, tensor_t *mask);	<-- zero inactive weights
float rigLMeasureSparsity(tensor_t *mask);	<-- count zeros / total

CONCEPT: BOOL Tensor for Sparsity Mask

Each weight tensor gets a companion BOOL tensor (mask) of identical shape.

mask->quantization->type = BOOL (or a 1-bit custom DType)

mask->data: packed bytes, 8 bits per byte

tensorBoolGet(mask, i) -> true=active, false=pruned

Memory cost: $N_{\text{weights}} / 8$ bytes (negligible vs weight tensor).

Example: Linear(256, 128) -> 32768 weights -> 4096 bytes for mask.

Weight tensor at FLOAT32: 131072 bytes. Mask overhead: 3.1%.

At 90% sparsity: only 3277 active weights out of 32768.

Effective computation: 90% fewer multiply-accumulates in forward pass.

Chapter 3 -- MinMax.h / MinMax.c

GAP IDENTIFIED

GAP: MinMax.h only provides global max/min with no index tracking.

findAbsMaxFloat exists but findAbsMinFloat does NOT exist.

MISSING: findAbsMinIndexFloat (for pruning -- find K weights with smallest |w|) and findAbsMaxIndexFloat (for growing -- find K gradients with largest |grad|).

Also missing: kSmallestAbsFloat, kLargestAbsFloat for batch prune/grow.

3.1 -- Existing Code (MinMax.h)

/* EXISTING -- no index tracking, no AbsMin */	
float findAbsMaxFloat(uint8_t *bytes, size_t n);	<-- ok for quantization
float findMaxFloat(uint8_t *bytes, size_t n);	
float findMinFloat(uint8_t *bytes, size_t n);	<-- signed min, not abs min
int32_t findMaxInt32(uint8_t *bytes, size_t n);	
int32_t findMinInt32(uint8_t *bytes, size_t n);	
/* MISSING for RigL: */	
/* findAbsMinFloat -- finds global abs min value */	
/* findAbsMinIndexFloat -- finds index of abs min (prune target) */	
/* findAbsMaxIndexFloat -- finds index of abs max (grow target) */	
/* findAbsThresholdFloat -- finds K-th smallest abs value */	

3.2 -- Gap Implementation

Add these functions to MinMax.h and MinMax.c:

/* MinMax.h additions */	
float findAbsMinFloat(uint8_t *bytes, size_t n);	<-- abs min value
size_t findAbsMinIndexFloat(float *arr, size_t n);	<-- index of abs min
size_t findAbsMaxIndexFloat(float *arr, size_t n);	<-- index of abs max
float findAbsKthSmallest(float *arr, tensor_t *mask,	
size_t n, size_t k, BOOL activeOnly);	<-- k-th smallest in active
float findAbsKthLargest(float *arr, tensor_t *mask,	
size_t n, size_t k, BOOL inactiveOnly);	<-- k-th largest in inactive
/* MinMax.c: findAbsMinFloat */	
float findAbsMinFloat(uint8_t *bytes, size_t n) {	
float *v = (float *)bytes;	
float mn = fabsf(v[0]);	<-- init with first

for (size_t i=1; i<n; i++) {	
float a = fabsf(v[i]);	
if (a < mn) mn = a;	<-- track minimum
}	
return mn;	
}	
/* findAbsKthSmallest: two-pass threshold for prune */	
float findAbsKthSmallest(float *arr, tensor_t *mask,	
size_t n, size_t k, BOOL activeOnly) {	
/* Pass 1: find global abs-min among active (as lower bound) */	
/* Pass 2+: repeatedly scan for next-smallest (selection) */	
float lastFound = -1.f;	<-- threshold from prev iteration
float kth = 0.f;	
for (size_t ki=0; ki<k; ki++) {	<-- find ki-th smallest
float cur = 1e38f;	<-- large init
for (size_t i=0; i<n; i++) {	
if (activeOnly && !tensorBoolGet(mask,i)) continue;	<-- skip inactive
float a = fabsf(arr[i]);	
if (a > lastFound && a < cur) cur = a;	<-- next smallest
}	
lastFound = cur; kth = cur;	
}	
return kth;	<-- threshold: prune all w <= kth
}	

CONCEPT: K-th Smallest: Selection-Sort Approach for Embedded MCU

A full sort of N weights (quicksort: $O(N \log N)$) requires $O(N)$ stack space for indices -- infeasible on 320KB MCU for $N=32768$.

Selection-sort approach: find the k-th smallest by k sequential scans.

Each scan: $O(N)$. Total: $O(k*N)$. For $k = \text{floor}(S * \text{nnz}(M))$ with $S=0.1$ (10% turnover),

$k \approx 0.1 * 0.1 * N = 0.01*N$. So $O(0.01*N * N) = O(0.01*N^2)$.

For $N=32768$: ~10M comparisons. At 216 MHz Cortex-M7: ~46ms per cycle.

Alternative (if stack permits): VLA of K indices for partial sort.

But selection-sort has $O(1)$ extra memory -- safe for MCU.

For the grow step: same algorithm but findAbsKthLargest on inactive weights.

Chapter 4 -- Matmul.h / Matmul.c + Arithmetic.h / Arithmetic.c

GAP IDENTIFIED

GAP (Matmul): matmulFloat32Tensors and matmulFloat32TensorsWithBias do not check sparsity mask. All weight elements are read even if pruned to zero.

GAP (Arithmetic): int32PointWiseArithmetic and float variants operate on ALL elements unconditionally -- no mask gating.

MISSING: matmulFloat32TensorsWithMask (skip zero-mask weights in inner loop), maskedFloatPointWiseArithmetic (skip inactive indices).

4.1 -- Matmul.c: Existing Inner Loop

/* EXISTING (simplified) -- no mask check */	
for (size_t rowIdx=0; rowIdx<aRows; rowIdx++) {	
for (size_t colIdx=0; colIdx<bCols; colIdx++) {	
float result = bias ? biasArr[colIdx] : 0.f;	
for (size_t k=0; k<aColumns; k++) {	
float aVal = aFloatArr[rowIdx*aColumns+k];	
float bVal = bFloatArr[k*bCols+colIdx];	<-- weight
result += aVal * bVal;	<-- NO MASK CHECK HERE
}	
outputArr[rowIdx*bCols+colIdx] = result;	
}	
}	

4.2 -- Gap Implementation: matmulFloat32TensorsWithMask

Add a masked matmul that skips the multiply when the weight mask bit is 0. The result is identical to multiplying by $W \cdot M$, but faster (no multiply-accumulate for pruned weights).

/* Add to Matmul.h: */	
void matmulFloat32TensorsWithMask(tensor_t *aTensor, tensor_t *bTensor,	<-- a=activations, b=weights
tensor_t *outputTensor, tensor_t *biasTensor, tensor_t *mask);	<-- BOOL mask for b (weights)
/* Matmul.c implementation: */	
void matmulFloat32TensorsWithMask(tensor_t *a, tensor_t *b, tensor_t *out, tensor_t *bias, tensor_t *mask) {	
float *aA=(float*)a->data, *bA=(float*)b->data;	
float *oA=(float*)out->data;	
float *bsA = bias ? (float*)bias->data : NULL;	
size_t aRows=getDimensionsByIndex(a,0);	

size_t aCol =getDimensionsByIndex(a,1);	
size_t bCol =getDimensionsByIndex(b,1);	
for (size_t r=0; r<aRows; r++) {	
for (size_t c=0; c<bCol; c++) {	
float acc = bsA ? bsA[c] : 0.f;	
for (size_t k=0; k<aCol; k++) {	
size_t bFlatIdx = k*bCol+c;	<-- flat weight index
if (mask && !tensorBoolGet(mask,bFlatIdx)) continue;	<-- SKIP if pruned
acc += aA[r*aCol+k] * bA[bFlatIdx];	<-- MAC only if active
}	
oA[r*bCol+c] = acc;	
}	
}	
}	

CONCEPT: Why Skip (not Multiply-by-Zero)

Multiplying by zero is mathematically correct: $\text{acc} += 0 * \text{input} = \text{acc}$.

Skipping the MAC entirely is an OPTIMIZATION for sparse hardware:

On ARM Cortex-M7 SIMD: a VMLA (vector multiply-accumulate) costs ~1-2 cycles.

Skipping it via branch: ~1 cycle (branch prediction on Cortex-M7 is excellent).

For 90% sparsity with N=32768: skip saves ~29491 MACs per forward pass.

At 216 MHz: ~136 microseconds saved per forward pass per layer.

However: branch misprediction overhead can erode gains for irregular sparsity.

Alternative: use a CSR (Compressed Sparse Row) format. Out of scope for ODT v1.

4.3 -- Arithmetic.c: masked element-wise

/* Add to Arithmetic.h: */	
void floatMaskedPointWise(tensor_t *a, tensor_t *b,	
floatElementArithmeticFunc_t fn, tensor_t *out,	
tensor_t *mask);	<-- elements where mask=0 are skipped
/* Arithmetic.c: */	
void floatMaskedPointWise(tensor_t *a, tensor_t *b,	
floatElementArithmeticFunc_t fn, tensor_t *out,	
tensor_t *mask) {	
size_t n = calcNumberOfElementsByTensor(a);	
float *aD=(float*)a->data, *bD=(float*)b->data, *oD=(float*)out->data;	

for (size_t i=0; i<n; i++) {	
if (mask && !tensorBoolGet(mask,i)) { oD[i]=0.f; continue; }	<-- pruned -> 0
oD[i] = fn(aD[i], bD[i]);	<-- active -> compute
}	
}	

Chapter 5 -- Linear.h / Linear.c / LinearApi.h / LinearApi.c

GAP IDENTIFIED

GAP (Linear.c): linearForwardFloat calls matmulFloat32TensorsWithBias without checking weights->sparsity. linearConfig_t has no mask field.

GAP (LinearApi.c): linearLayerInit / linearLayerInitOwning do not allocate or initialize a BOOL mask tensor for the weights parameter.

MISSING: mask field in linearConfig_t, masked forward path, api init with mask.

5.1 -- Linear.h: Add mask to linearConfig_t

/* Linear.h: existing linearConfig_t (simplified) */	
typedef struct linearConfig {	
parameter_t *weights;	
parameter_t *bias;	
quantization_t *forwardQ;	
quantization_t *backwardQ;	
BOOL ownsQuantizations;	
/* GAP: no sparsity mask here */	
} linearConfig_t;	
/* Gap implementation: add mask field */	
typedef struct linearConfig {	
parameter_t *weights;	
parameter_t *bias;	
quantization_t *forwardQ;	
quantization_t *backwardQ;	
BOOL ownsQuantizations;	
tensor_t *weightMask;	<-- BOOL tensor, same shape as weights
rigLSparsityConfig_t *sparsityConfig;	<-- NULL if dense
} linearConfig_t;	

5.2 -- Linear.c: Masked Forward Pass

/* EXISTING (no mask) */	
static void linearForwardFloat(linearConfig_t *c,	
tensor_t *input, tensor_t *output) {	
tensor_t *W = getParamFromParameter(c->weights);	
tensor_t *b = c->bias ?	
getParamFromParameter(c->bias) : NULL;	
matmulFloat32TensorsWithBias(input, W, output, b);	<-- no mask!
}	
/* Gap implementation: check weightMask */	

static void linearForwardFloat(linearConfig_t *c,	
tensor_t *input, tensor_t *output) {	
tensor_t *W = getParamFromParameter(c->weights);	
tensor_t *b = c->bias ?	
getParamFromParameter(c->bias) : NULL;	
if (c->weightMask != NULL) {	<-- sparse path
matmulFloat32TensorsWithMask(input, W, output, b, c->weightMask);	
} else {	<-- dense path (unchanged)
matmulFloat32TensorsWithBias(input, W, output, b);	
}	
}	

CONCEPT: Why Dense Path Unchanged

Adding a NULL check for weightMask means:

- Dense layers (weightMask==NULL): exact same code path as before. Zero overhead.
- Sparse layers (weightMask!=NULL): routed to masked matmul.

This is the open-closed principle: existing functionality unchanged, new functionality added via extension.

5.3 -- Linear.c: Backward with Mask (weight grad re-masking)

The backward pass computes weight gradients for ALL weights. After the optimizer step, pruned weight grads should not cause parameter updates. Re-masking in the optimizer handles this (Chapter 9).

/* linearCalcWeightGradsFloat32: no change needed */	
/* Grads are computed for ALL weights including pruned ones. */	
/* This is INTENTIONAL for RigL: pruned weights get non-zero grad,	
which is used in the GROW step to select which to reactivate. */	
/* After grow, selected weights are unmasked and get nonzero values from grad. */	
/* The optimizer re-masking step (Chapter 9) zeros out any param updates to pruned weights. */	

WARNING: Gradient for Pruned Weights

When weight w_i is masked to 0: forward contribution = $0 * \text{input}_i = 0$. Backward: $dL/dw_i = \text{input}_i * dL/d_{\text{output}_j}$. This is NONZERO even though $w_i=0$. This gradient tells us: if w_i became active, how much would the loss decrease? RigL uses exactly this gradient for the GROW step. Do NOT zero it out!

5.4 -- LinearApi.c: Mask initialization

/* Add to linearLayerInit: */	
-------------------------------	--

void linearLayerInitSparse(linearConfig_t *c, float targetSparsity,	
tensor_t *maskBuffer, uint8_t *maskData,	<-- caller-allocated
size_t updateFreq, size_t numCycles,	
rigLSparsityConfig_t *sparsityCfg) {	
/* Initialize mask to all-1 (fully dense initially) */	
size_t n = calcNumberOfElementsByParameter(c->weights);	
setTensorValues(maskBuffer, maskData, c->weights->param->shape, getBoolQuantization(), NULL);	
for (size_t i=0; i<n; i++) tensorBoolSet(maskBuffer, i, true);	<-- all active
initRigLSparsityConfig(sparsityCfg, maskBuffer, targetSparsity, updateFreq, numCycles);	
c->weightMask = maskBuffer;	
c->sparsityConfig = sparsityCfg;	
}	

Chapter 6 -- Conv1d.h/.c and Conv1dTransposed.h/.c

GAP IDENTIFIED

GAP (Conv1d.c): conv1dKernelFloat32 accumulates over all (ic, k) without checking whether weight W[oc][ic][k] is active. conv1dConfig_t has no mask field.

GAP (Conv1dTransposed.c): scatter-based forward writes to output even for pruned weights. No mask field in conv1dTransposedConfig_t.

MISSING: weightMask tensor in both configs; mask check in inner loop.

6.1 -- Conv1d.h: Add weightMask to conv1dConfig_t

typedef struct conv1dConfig {	
parameter_t *weights;	
parameter_t *bias;	
size_t inChannels, outChannels, kernelSize;	
size_t stride, padding, groups;	
size_t inChPerGroup, outChPerGroup;	
quantization_t *forwardQ, *backwardQ;	
BOOL ownsQuantizations;	
/* GAP: add these two fields */	
tensor_t *weightMask;	<-- BOOL tensor shape [outCh, inChPerGroup, K]
rigLSparsityConfig_t *sparsityConfig;	<-- NULL = dense layer
} conv1dConfig_t;	

6.2 -- Conv1d.c: Masked Inner Loop

The weight tensor for Conv1d has shape [outCh, inChPerGroup, kernelSize]. The mask has the same shape. At each (oc, ic, k), check mask before MAC.

/* EXISTING inner loop (simplified) */	
for (size_t ic=0; ic<inChPerGroup; ic++) {	
for (size_t k=0; k<K; k++) {	
size_t wIdx = (ocAbs*inChPerGroup+ic)*K+k;	<-- flat weight index
float wVal = wArr[wIdx];	<-- no mask check
acc += inArr[...][inPos] * wVal;	
}	
}	
/* Gap implementation: add mask check */	
for (size_t ic=0; ic<inChPerGroup; ic++) {	
for (size_t k=0; k<K; k++) {	
size_t wIdx = (ocAbs*inChPerGroup+ic)*K+k;	<-- flat weight index
if (c->weightMask && !tensorBoolGet(c->weightMask,wIdx)) continue;	<-- SKIP pruned
acc += inArr[...][inPos] * wArr[wIdx];	<-- active only

```
}
}
```

CONCEPT: Sparsity in Conv1d vs Linear

Linear: mask shape [outFeatures, inFeatures] -- one mask bit per connection.

Conv1d: mask shape [outChannels, inChPerGroup, kernelSize] -- one bit per weight.

For grouped convolution: each output group has outChPerGroup output channels and inChPerGroup input channels. The mask must be consistent with grouping.

Pruning structured (e.g. entire kernels $W[oc][*][*]$) is more hardware-friendly

but RigL uses unstructured (per-weight) sparsity by default.

For STM32F746 without sparse math hardware: unstructured sparsity only helps if we skip MAC explicitly (with the mask check above). Structured sparsity allows simpler channel pruning with standard dense kernels.

6.3 -- Conv1dTransposed.c: Masked Scatter

```
/* EXISTING scatter (simplified) */
for (size_t ic ...) for (size_t inPos ...) {
for (size_t oc ...) for (size_t k ...) {
size_t wIdx = (ocAbs*inChPerGroup+ic)*K+k;
outArr[b][ocAbs][inPos*stride+k] +=
inArr[b][icAbs][inPos] * wArr[wIdx];
}
}
}

/* Gap implementation */
for (size_t ic ...) for (size_t inPos ...) {
for (size_t oc ...) for (size_t k ...) {
size_t wIdx = (ocAbs*inChPerGroup+ic)*K+k;
if (c->weightMask &&
!tensorBoolGet(c->weightMask,wIdx)) continue;
outArr[b][ocAbs][inPos*stride+k] +=
inArr[b][icAbs][inPos] * wArr[wIdx];
}
}
}
```

<-- no mask

<-- skip pruned

Chapter 7 -- Optimizer.h / Sgd.h / Sgd.c

GAP IDENTIFIED

GAP (Sgd.c): sgdStepFloat updates ALL parameters unconditionally, even pruned weights. After the optimizer step, pruned weights are no longer zero (they drift). This invalidates the sparsity mask.

GAP (Optimizer.h): optimizer_t has no reference to layer sparsity configs.

MISSING: re-masking step after SGD update to zero out drifted pruned weights.

7.1 -- Sgd.c: Existing Update (no re-masking)

/* EXISTING sgdStepFloat -- no mask awareness */	
for (size_t e=0; e<n; e++) {	
float grad = gradArr[e] + sgd->weightDecay * dataArr[e];	
dataArr[e] -= sgd->learningRate * grad;	<-- updates ALL weights
/* BUG for RigL: pruned weight[e] was 0, now it's nonzero! sparsity violated. */	
}	

7.2 -- Gap Implementation: Re-mask after SGD step

After updating weights, re-apply the sparsity mask. This zeros out any drift accumulated in pruned weight positions.

/* Add to Sgd.h: */	
void sgdStepWithReMask(optimizer_t *optim,	
tensor_t **masks, size_t numMasks);	<-- parallel array of masks
/* Sgd.c implementation: */	
static void sgdStepFloatWithMask(optimizer_t *optim,	
tensor_t **masks) {	
sgd_t *sgd = (sgd_t *)optim->impl;	
for (size_t si=0; si<optim->sizeStates; si++) {	
parameter_t *param = optim->parameter[si];	
size_t n = calcNumberOfElementsByParameter(param);	
float *gradArr = (float *)param->grad->data;	
float *dataArr = (float *)param->param->data;	
tensor_t *mask = masks ? masks[si] : NULL;	<-- per-param mask
for (size_t e=0; e<n; e++) {	
/* Skip update for inactive weights -- grad is non-zero (used in grow) */	
/* but we still ZERO the weight to prevent drift: */	
float grad = gradArr[e] + sgd->weightDecay * dataArr[e];	
dataArr[e] -= sgd->learningRate * grad;	<-- update

/* Re-mask: force pruned weights back to zero */	
if (mask && !tensorBoolGet(mask,e)) {	<-- if pruned:
dataArr[e] = 0.f;	<-- zero out drift
}	
}	
}	
}	

CONCEPT: Why Re-Mask, Not Skip-Update?

Option A: Skip update entirely for pruned weights (if !mask[e], continue).

Problem: weight[e] stays at 0 (no drift). BUT: the gradient for the GROW step must not be consumed by the optimizer. This requires tracking separately.

Option B: Update ALL weights, then re-mask (set weight[e]=0 if mask[e]==0).

Pro: Simpler -- one loop, no branching needed for gradient tracking.

Con: Slightly wasteful (compute update for pruned weight then discard).

Option B is the RigL reference implementation approach. The computation cost for pruned updates is dwarfed by forward pass savings at 90% sparsity.

Key insight: the gradient for a pruned weight is STILL nonzero (see Chapter 5). After re-masking, that gradient is preserved in param->grad for the GROW step BEFORE sgdZeroGrad is called.

CONCEPT: Optimizer State and Masks

optimizer_t tracks parameter_t** (parameters) but no mask references.

To support masked SGD, extend optimizer_t:

```
typedef struct optimizer {
```

...existing fields...

```
tensor_t **paramMasks; /* parallel array: masks[i] for parameter[i] */
```

```
size_t numMasks; /* should equal sizeStates */
```

```
} optimizer_t;
```

NULL in paramMasks[i] = dense parameter (no mask).

Non-NULL = sparse parameter; use mask in re-masking step.

Chapter 8 -- TrainingEpochDefault.c / CalculateGradsSequential.c

GAP IDENTIFIED

GAP (TrainingEpochDefault.c): The training loop calls step -> zero-grad with no RigL update step. There is no global step counter and no call to prune/grow weights periodically.

GAP (CalculateGradsSequential.c): Gradient computation is correct for RigL (inactive weights still get nonzero grad) but there is no hook to accumulate or save the raw gradient before zero-grad clears it.

MISSING: rigLStep() call in training loop after backward and before zero-grad; global step counter; per-model sparsity context.

8.1 -- TrainingEpochDefault.c: Existing Loop

/* EXISTING (simplified) */	
for (size_t i=0; i<numberOfBatches; i++) {	
batch_t *batch = dataLoader->getBatch(dataLoader, i);	
totalLoss += trainingBatchDefault(model, ..., calculateGradsFn);	<-- backward done
/* --- GAP: no rigLStep here --- */	
optimFns.step(optimizer);	<-- parameter update
/* --- GAP: no re-masking here --- */	
optimFns.zero(optimizer);	<-- grad zeroed -- DESTROY grad info!
freeBatch(batch);	
}	

8.2 -- Gap Implementation: RigL-Aware Training Epoch

The RigL step must happen BETWEEN backward (grads computed) and zero-grad (grads cleared). This is the only window where pruned-weight gradients are available.

/* Add to TrainingEpochDefault.h: */	
float trainingEpochRigL(	
layer_t **model, size_t modelSize,	
lossConfig_t lossConfig, dataLoader_t *dataLoader,	
optimizer_t *optimizer,	
calculateGradsFn_t calculateGradsFn,	
rigLConfig_t *rigLCfg,	<-- RigL schedule config
reduction_t forwardReduction);	
/* Implementation (pseudo) */	
for (size_t i=0; i<numBatches; i++) {	
batch_t *batch = dataLoader->getBatch(dataLoader, i);	
totalLoss += trainingBatchDefault(model, modelSize, lossConfig,	

batch, calculateGradsFn, forwardReduction);	
/* 1. RigL: prune+grow if time (uses grad before zero) */	
if (rigLCfg != NULL) {	
rigLModelStep(model, modelSize, rigLCfg);	<-- prune+grow all layers
}	
/* 2. Optimizer step (update weights) */	
optimFns.step(optimizer);	
/* 3. Re-mask: zero out drift in pruned weights */	
if (rigLCfg != NULL) {	
rigLReMaskModel(model, modelSize);	<-- force pruned -> 0
}	
/* 4. Zero grad (after RigL used them) */	
optimFns.zero(optimizer);	
freeBatch(batch);	
}	

CONCEPT: Critical Ordering: backward -> RigL -> step -> re-mask -> zero-grad

1. backward(): Computes grad for ALL weights (active and pruned).

2. rigLStep(): Uses grad[inactive] to find which to GROW.

Uses |w[active]| to find which to PRUNE.

Updates mask M. Newly grown: M[i]=1, w[i] unchanged (nonzero from grad).

Newly pruned: M[i]=0, w[i]=0.

3. step(): SGD updates w[i] for all i.

4. re-mask(): w[i] = 0 for all i where M[i]==0. Clears drift.

5. zero-grad(): Clears all gradients. Must be LAST.

If zero-grad() is called before rigLStep(): grow step has no signal.

If re-mask() is skipped: pruned weights accumulate SGD updates -> sparsity violated.

8.3 -- rigLConfig_t: Training Schedule

/* Add to RigL.h: */	
typedef struct rigLConfig {	
size_t updateFreq; /* steps between prune+grow (T in paper) */	
size_t numCycles; /* total cycles (C in paper) */	
float turnoverFrac; /* fraction of active weights to swap (Delta in paper) */	
size_t globalStep; /* incremented every gradient step */	

size_t cyclesDone; /* incremented every prune+grow cycle */	
} rigLConfig_t;	
/* rigLModelStep: check schedule, then update each sparse layer */	
void rigLModelStep(layer_t **model, size_t N, rigLConfig_t *cfg) {	
cfg->globalStep++;	<-- count every step
if (cfg->cyclesDone >= cfg->numCycles) return;	<-- mask frozen
if (cfg->globalStep % cfg->updateFreq != 0) return;	<-- not time yet
for (size_t i=0; i<N; i++) {	
if (model[i]->type == LINEAR)	
rigLLinearStep(model[i]->config->linear, cfg);	
else if (model[i]->type == CONV1D)	
rigLConv1dStep(model[i]->config->conv1d, cfg);	
/* etc for other sparse layer types */	
}	
cfg->cyclesDone++;	
}	

Chapter 9 -- Layer.h / Layer.c, Serialize/Deserialize, Other Files

GAP IDENTIFIED

GAP (Layer.h): layerConfig_t union has no sparsity pointer. Adding a sparsity field to each config individually (Chapters 5/6) is cleaner than a union-level field, but Layer.h needs a way to query 'does this layer have a sparse mask?'.

GAP (Serialize.h/c): BOOL mask tensors are not serialized. A saved sparse model would load as dense (all weights nonzero) on restart.

GAP (Optimizer.h): calcTotalNumberOfStates counts parameters but not mask tensors. GAP (Distributions.c): No mask-respecting initialization (e.g., initialize only active weights).

9.1 -- Layer.h: Query Sparsity

/* Add to Layer.h: */	
/* Returns mask tensor for a layer, or NULL if dense */	
tensor_t *getLayerWeightMask(layer_t *layer);	
/* Layer.c implementation: */	
tensor_t *getLayerWeightMask(layer_t *layer) {	
switch (layer->type) {	
case LINEAR:	
return layer->config->linear->weightMask;	
case CONV1D:	
return layer->config->conv1d->weightMask;	
case CONV1D_TRANSPOSED:	
return	
layer->config->conv1dTransposed->weightMask;	
default:	
return NULL;	<-- no weights to mask
}	
}	

9.2 -- Serialize.h/c: Mask Serialization

When saving a sparse model, the mask must be serialized alongside weights. Otherwise, loading restores weights but with mask=all-ones (dense). The next training step would compute wrong sparsity.

/* Add to Serialize.h: */	
void serializeLayerMasks(layer_t **model, size_t modelSize,	
uint8_t *buffer, size_t bufSize);	
/* Add to Deserialize.h: */	
void deserializeLayerMasks(layer_t **model, size_t modelSize,	

```
uint8_t const *buffer);

/* Format: [layer_idx][mask_n_bytes][mask_data...]
for each sparse layer */

/* Dense layers are skipped (no mask to save) */
```

WARNING: Mask Must Be Saved With Weights

If weights are serialized but mask is not: on reload, all weights are loaded. However the mask bits are all 1 (initialized by default). The model is now DENSE. First forward pass will use all weights including those that should be pruned. Accuracy drops. Then the first RigL step will re-prune, but it will prune randomly (not the same weights as before), losing convergence progress.

9.3 -- Distributions.c: Sparse-Aware Initialization

When a grown weight transitions from inactive (0) to active, it needs an initial value. Options:

```
/* Option 1: Leave at 0 when grown */
/* mask[i] = 1 (now active), weight[i] = 0 (no
special init) */
/* SGD will update it next step via gradient */
/* RigL paper uses this: newly grown weights start
at 0 */

/* Option 2: Initialize with Kaiming when grown */
void rigLInitGrownWeight(float *arr, size_t idx,
size_t fanIn) {
arr[idx] = kaimingUniform(1.f, fanIn);
}
/* This gives stronger initial gradient signal for
newly grown weights */
```

CONCEPT: RigL Paper: Newly Grown Weights Start at 0

Evci et al. (2020) initialize newly grown weights to 0. This is intentional:

- The weight was inactive (0) and accumulated a nonzero gradient signal.
- Setting it to 0 and letting SGD update it avoids a cold-start problem.
- The gradient at step T tells SGD which direction to move w from 0.

Alternative (Mostafa & Wang): set newly grown weight to its gradient value.

$w_{\text{new}} = lr * \text{grad}$ (one gradient step from 0)

This can accelerate early convergence of newly grown weights.

For ODT/STM32 implementation: start at 0 (simpler, matches original paper).

9.4 -- Other Files: No Significant Gap

The following files require no modification for RigL:

DTypes.h/c: byte read/write utilities; no RigL gap.

TensorConversion.h/c: convertTensor between dtypes; mask is BOOL dtype, not converted during training.

Quantization.h/c: SYM_INT32 quantization; masks are always BOOL (unquantized).

LossFunction.h/c, CrossEntropy.h/c, MSE.h/c: loss functions unchanged.

RNG.h/c, Bernoulli.h/c: random number generators unchanged (used by Dropout, not RigL).

DataLoader.h/c, NPYLoader.h/c: data loading unchanged.

CSVHelper.h/c: CSV utilities unchanged.

userApi layer/*.c: API wrappers need mask init (similar to LinearApi, Chapter 5).

Chapter 10 -- RigL.h: Complete Header Implementation

IMPLEMENTATION

RigL.h is a NEW file (does not exist in ODT library yet).

It must be created at: src/rigl/include/RigL.h

It declares all types and functions needed to integrate RigL into ODT.

/* ===== =====	
* RigL.h -- Sparse training via RigLess (Evci et al. 2020)	
* For: STM32 Nucleo-F746ZG (Cortex-M7, 320KB SRAM)	
* ===== ===== */	
#ifndef ODT_RIGL_H	<-- include guard
#define ODT_RIGL_H	
#include <stdbool.h>	
#include <stddef.h>	
#include <stdint.h>	
#include "Tensor.h"	<-- tensor_t, parameter_t
#include "Layer.h"	<-- layer_t, layerType_t
#include "Linear.h"	<-- linearConfig_t
#include "Conv1d.h"	<-- conv1dConfig_t
/* ---- RigL Sparsity Config (fills empty sparsityConfig stub) ---- */	
typedef struct rigLSparsityConfig {	
tensor_t *mask; /* BOOL tensor, same shape as weights */	
float targetSparsity; /* e.g. 0.90 = 90% inactive weights */	
float turnoverFrac; /* fraction of active weights swapped per cycle */	
size_t updateFreq; /* gradient steps between prune+grow */	
size_t numCycles; /* total prune+grow cycles (then mask frozen) */	
size_t stepCount; /* internal: steps since last update */	
size_t cycleCount; /* internal: completed cycles */	
} rigLSparsityConfig_t;	
/* ---- RigL Model-Level Schedule ---- */	
typedef struct rigLModelConfig {	
size_t updateFreq; /* T: steps between mask updates */	

size_t numCycles; /* C: total mask update cycles */	
float turnoverFrac; /* Delta: fraction of weights swapped */	
size_t globalStep; /* internal: incremented every grad step */	
size_t cyclesDone; /* internal: completed cycles */	
} rigLModelConfig_t;	
/* ---- Initialization ---- */	
void rigLInitSparsityConfig(	
rigLSparsityConfig_t *cfg,	<-- config to fill
tensor_t *mask,	<-- pre-allocated BOOL tensor
float targetSparsity,	<-- e.g. 0.90
float turnoverFrac,	<-- e.g. 0.20 (20% weights swapped/cycle)
size_t updateFreq,	<-- e.g. 100 (every 100 steps)
size_t numCycles);	<-- e.g. 10 (10 prune+grow cycles total)
void rigLInitModelConfig(	
rigLModelConfig_t *cfg,	
size_t updateFreq, size_t numCycles, float turnoverFrac);	
/* ---- Mask Operations ---- */	
/* Initialize mask from current weights: prune to targetSparsity fraction */	
void rigLInitMaskFromWeights(tensor_t *mask,	
tensor_t *weights,	
float targetSparsity);	<-- prune smallest w until sparsity reached
/* Apply mask to weights: w[i] = 0 if mask[i]==0 */	
void rigLApplyMask(tensor_t *weights, tensor_t *mask);	
/* Re-apply mask after optimizer step (zero drift) */	
void rigLReMask(tensor_t *weights, tensor_t *mask);	<-- identical to ApplyMask
/* Count active (mask==1) weights */	
size_t rigLCountActive(tensor_t *mask);	
/* Measure current sparsity: inactive / total */	
float rigLMeasureSparsity(tensor_t *mask);	
/* ---- Core Update: Prune + Grow ---- */	
void rigLUpdateMask(	
tensor_t *weights,	<-- weight tensor (read w for prune)
tensor_t *grads,	<-- gradient tensor (read grad for grow)
tensor_t *mask,	<-- BOOL mask tensor (modified in-place)

size_t numToSwap);	<-- K = number of weights to prune+grow
/* ---- Layer-Level Step (called per sparse layer) ---- */	
void rigLLinearStep(linearConfig_t *cfg, size_t numToSwap);	
void rigLConv1dStep(conv1dConfig_t *cfg, size_t numToSwap);	
/* ---- Model-Level Step (called once per gradient step) ---- */	
void rigLModelStep(layer_t **model, size_t modelSize, rigLModelConfig_t *cfg);	
/* Re-mask all sparse layers in model after optimizer step */	
void rigLReMaskModel(layer_t **model, size_t modelSize);	
#endif /* ODT_RIGL_H */	

CONCEPT: RigL.h Design Decisions

1. rigLSparsityConfig_t is a per-layer struct (not per-model).

Each sparse layer owns its mask and schedule independently.

This allows layers with different target sparsities.

2. rigLModelConfig_t is a model-level schedule (shared T, C, Delta).

rigLModelStep() uses the model config to decide WHEN to update,
then calls per-layer update with numToSwap = floor(Delta * nnz(M)).

3. Two mask functions with different names but identical implementation:

rigLApplyMask: used at init (first application of mask to weights)

rigLReMask: used after optimizer step (re-zeroing drift)

Having two names makes calling code more readable.

4. rigLUpdateMask takes numToSwap (not targetSparsity) for flexibility.

The caller computes numToSwap = floor(turnoverFrac * nnz(mask)).

Chapter 11 -- RigL.c: Complete Implementation

IMPLEMENTATION

RigL.c is a NEW file (does not exist in ODT library yet).

Create at: src/rigl/RigL.c

This file implements ALL functions declared in RigL.h.

11.1 -- File Header and Includes

#define SOURCE_FILE "RIGL"	<-- for PRINT_ERROR macro
#include <math.h>	<-- fabsf
#include <stddef.h>	<-- size_t
#include <stdlib.h>	<-- exit
#include <string.h>	<-- memset
#include "Common.h"	<-- PRINT_ERROR, reserveMemory
#include "Conv1d.h"	<-- conv1dConfig_t
#include "Layer.h"	<-- layer_t, layerType_t
#include "Linear.h"	<-- linearConfig_t
#include "RigL.h"	<-- own header
#include "Tensor.h"	<-- tensor_t, tensorBoolGet/Set
/* ===== ==== */	
/* SECTION 1: Initialization */	
/* ===== ==== */	

11.2 -- rigLInitSparsityConfig

void rigLInitSparsityConfig(rigLSparsityConfig_t *cfg, tensor_t *mask, float targetSparsity, float turnoverFrac, size_t updateFreq, size_t numCycles) { if (targetSparsity < 0.f targetSparsity >= 1.f) { PRINT_ERROR("targetSparsity must be in [0,1)"); exit(1); } if (turnoverFrac <= 0.f turnoverFrac > 1.f) { PRINT_ERROR("turnoverFrac must be in (0,1]"); exit(1); }	
---	--

}	
if (updateFreq == 0) {	
PRINT_ERROR("updateFreq must be > 0");	
exit(1);	
}	
cfg->mask = mask;	<-- store mask pointer
cfg->targetSparsity = targetSparsity;	
cfg->turnoverFrac = turnoverFrac;	
cfg->updateFreq = updateFreq;	
cfg->numCycles = numCycles;	
cfg->stepCount = 0;	<-- reset counters
cfg->cycleCount = 0;	
}	

CONCEPT: Validation in initConfig

targetSparsity >= 1.0 is invalid: would require all weights pruned, leaving zero active weights (model outputs always 0).

turnoverFrac = 0 is invalid: would never swap any weights (mask never changes).

turnoverFrac = 1.0 is valid: swap ALL active weights each cycle (extreme churn).

updateFreq = 0: division by zero when checking stepCount % updateFreq.

Typical values: targetSparsity=0.9, turnoverFrac=0.2, updateFreq=100, numCycles=10.

11.3 -- rigLInitMaskFromWeights: Initial Pruning

At the start of RigL training, the mask is initialized by pruning the targetSparsity fraction of weights with smallest $|w|$.

void rigLInitMaskFromWeights(tensor_t *mask, tensor_t *weights, float targetSparsity) {	
size_t n = calcNumberOfElementsByTensor(weights);	<-- total weights
float *wArr = (float *)weights->data;	<-- assume FLOAT32
/* Step 1: Set all mask bits to 1 (fully active) */	
for (size_t i=0; i<n; i++) tensorBoolSet(mask, i, true);	
/* Step 2: Find threshold = targetSparsity-th percentile of $ w $ */	
size_t numToPrune = (size_t)(targetSparsity * (float)n);	
if (numToPrune == 0) return;	<-- nothing to prune
/* Step 3: Selection -- find K-th smallest $ w $ using sequential scan */	

float threshold = rigLFindKthSmallestAbs(wArr, n, numToPrune);	
/* Step 4: Prune all w <= threshold */	
size_t pruned = 0;	
for (size_t i=0; i<n; i++) {	
if (pruned < numToPrune && fabsf(wArr[i]) <= threshold) {	
tensorBoolSet(mask, i, false);	<-- deactivate
wArr[i] = 0.f;	<-- zero out
pruned++;	
}	
}	
}	

11.4 -- rigLFindKthSmallestAbs: Selection Algorithm (static helper)

static float rigLFindKthSmallestAbs(float *arr, size_t n, size_t k) {	
/* Selection sort style: O(k*n) -- safe for embedded MCU, O(1) extra */	
float lastFound = -1.f;	<-- threshold from previous iteration
float kth = 0.f;	
for (size_t ki=0; ki<k; ki++) {	<-- find the ki-th smallest
float cur = 1e38f;	<-- init with large sentinel
for (size_t i=0; i<n; i++) {	
float a = fabsf(arr[i]);	
if (a > lastFound && a < cur) cur = a;	<-- next smallest
}	
if (cur == 1e38f) break;	<-- fewer than k distinct values
lastFound = cur;	
kth = cur;	
}	
return kth;	<-- the k-th smallest abs value
}	

CONCEPT: Selection Algorithm: How It Works

Goal: find the K-th smallest value in array without sorting.

Iteration 0: scan all n values, find the global minimum abs value -> kth_0.

Iteration 1: scan all n values, find the smallest abs value > kth_0 -> kth_1.

Iteration k-1: scan all n values, find the smallest abs value > kth_{k-2}.

After K iterations: kth is the K-th smallest abs value.

Then: prune all i where |w[i]| <= kth (up to K elements).

Cost: $O(K*N)$ time, $O(1)$ space. For $K = 0.9*N$ (extreme case): $O(0.9*N^2)$.

For $K = 0.2 * 0.1 * N = 0.02*N$ (typical turnover): $O(0.02*N^2)$.

$N=8192$: $\sim 1.3M$ comparisons per cycle. At 216MHz: $\sim 6ms$. Acceptable.

Faster alternative: QuickSelect $O(N)$ average, but requires $O(N)$ stack space.

11.5 -- rigLApplyMask / rigLReMask

void rigLApplyMask(tensor_t *weights, tensor_t *mask) {	
size_t n = calcNumberOfElementsByTensor(weights);	
float *w = (float *)weights->data;	
for (size_t i=0; i<n; i++) {	
if (!tensorBoolGet(mask, i)) {	<-- if pruned:
w[i] = 0.f;	<-- zero out
}	
}	
}	
/* rigLReMask: identical implementation, different semantic name */	
void rigLReMask(tensor_t *weights, tensor_t *mask)	
{	
rigLApplyMask(weights, mask);	<-- same logic
}	
size_t rigLCountActive(tensor_t *mask) {	
size_t n = calcNumberOfElementsByTensor(mask);	
size_t count = 0;	
for (size_t i=0; i<n; i++)	
if (tensorBoolGet(mask, i)) count++;	
return count;	
}	
float rigLMeasureSparsity(tensor_t *mask) {	
size_t n = calcNumberOfElementsByTensor(mask);	
size_t active = rigLCountActive(mask);	
return 1.f - (float)active / (float)n;	
}	

11.6 -- rigLUpdateMask: The Core Prune+Grow Step

This is the heart of RigL. It prunes K active weights with smallest $|w|$, then grows K inactive weights with largest $|\text{grad}|$.

void rigLUpdateMask(	
tensor_t *weights, tensor_t *grads,	
tensor_t *mask, size_t numToSwap) {	

size_t n = calcNumberOfElementsByTensor(weights);	
float *wArr = (float *)weights->data;	
float *gArr = (float *)grads->data;	
if (numToSwap == 0) return;	<-- nothing to do
/* ===== PRUNE STEP ===== */	
/* Find K-th smallest w among ACTIVE weights */	
float pruneThresh = rigLFindKthSmallestAbsActive(wArr, mask, n, numToSwap);	
/* Prune: set mask=0 and w=0 for w <= thresh (active only) */	
size_t pruned = 0;	
for (size_t i=0; i<n; i++) {	
if (!tensorBoolGet(mask, i)) continue;	<-- skip already-inactive
if (pruned < numToSwap && fabsf(wArr[i]) <= pruneThresh) {	
tensorBoolSet(mask, i, false);	<-- deactivate
wArr[i] = 0.f;	<-- zero weight
pruned++;	
}	
}	
/* ===== GROW STEP ===== */	
/* Find K-th largest grad among INACTIVE weights */	
float growThresh = rigLFindKthLargestAbsInactive(gArr, mask, n, numToSwap);	
/* Grow: set mask=1 and keep w=0 for g >= thresh (inactive only) */	
size_t grown = 0;	
for (size_t i=0; i<n; i++) {	
if (tensorBoolGet(mask, i)) continue;	<-- skip active
if (grown < numToSwap && fabsf(gArr[i]) >= growThresh) {	
tensorBoolSet(mask, i, true);	<-- activate
/* weight stays 0: SGD will update from nonzero grad */	
grown++;	
}	
}	
}	

CONCEPT: Prune + Grow: Maintaining Constant Sparsity

After init: $\text{nnz}(\text{mask}) = n * (1 - \text{targetSparsity})$ active weights.

After PRUNE: $\text{nnz}(\text{mask})$ decreases by numToSwap.

After GROW: $\text{nnz}(\text{mask})$ increases by numToSwap .

Net change: 0. Sparsity is exactly preserved each cycle.

This is the key property of RigL: constant sparsity throughout training.

$\text{numToSwap} = \text{floor}(\text{turnoverFrac} * \text{nnz}(\text{mask}))$.

Example: $\text{nnz}=3277$ (10% active from 32768), $\text{turnoverFrac}=0.20$:

$\text{numToSwap} = \text{floor}(0.20 * 3277) = 655$ weights swapped per cycle.

11.7 -- `rigLFindKthSmallestAbsActive` (static helper)

Like `findKthSmallestAbs` but considers ONLY active weights ($\text{mask}==1$).

<code>static float rigLFindKthSmallestAbsActive(</code>	
<code>float *arr, tensor_t *mask, size_t n, size_t k) {</code>	
<code>float lastFound = -1.f;</code>	
<code>float kth = 0.f;</code>	
<code>for (size_t ki=0; ki<k; ki++) {</code>	
<code>float cur = 1e38f;</code>	
<code>for (size_t i=0; i<n; i++) {</code>	
<code>if (!tensorBoolGet(mask,i)) continue;</code>	<-- skip inactive
<code>float a = fabsf(arr[i]);</code>	
<code>if (a > lastFound && a < cur) cur = a;</code>	
<code>}</code>	
<code>if (cur == 1e38f) break;</code>	
<code>lastFound = cur; kth = cur;</code>	
<code>}</code>	
<code>return kth;</code>	
<code>}</code>	
<code>static float rigLFindKthLargestAbsInactive(</code>	
<code>float *arr, tensor_t *mask, size_t n, size_t k) {</code>	
<code>float lastFound = 1e38f;</code>	<-- init large for largest scan
<code>float kth = 0.f;</code>	
<code>for (size_t ki=0; ki<k; ki++) {</code>	
<code>float cur = -1.f;</code>	<-- init small
<code>for (size_t i=0; i<n; i++) {</code>	
<code>if (tensorBoolGet(mask,i)) continue;</code>	<-- skip active
<code>float a = fabsf(arr[i]);</code>	
<code>if (a < lastFound && a > cur) cur = a;</code>	<-- next largest
<code>}</code>	
<code>if (cur == -1.f) break;</code>	
<code>lastFound = cur; kth = cur;</code>	
<code>}</code>	
<code>return kth;</code>	

```
}
```

WARNING: Tie-Breaking in Selection

When multiple weights have the same abs value at the threshold, the scan prunes/grows the first K encountered (scan order = flat tensor index). This is deterministic but not random. If exact tie-breaking matters, add a secondary tiebreaker (e.g., gradient magnitude for prune ties). For most practical cases, ties are rare and the simple approach is sufficient.

11.8 -- rigLLinearStep and rigLConv1dStep

void rigLLinearStep(linearConfig_t *cfg, size_t numToSwap) {	
if (cfg->weightMask == NULL) return;	<-- dense layer -- skip
if (cfg->sparsityConfig == NULL) return;	
tensor_t *W = getParamFromParameter(cfg->weights);	
tensor_t *G = getGradFromParameter(cfg->weights);	
tensor_t *M = cfg->weightMask;	
rigLUpdateMask(W, G, M, numToSwap);	<-- core prune+grow
}	
void rigLConv1dStep(conv1dConfig_t *cfg, size_t numToSwap) {	
if (cfg->weightMask == NULL) return;	
tensor_t *W = getParamFromParameter(cfg->weights);	
tensor_t *G = getGradFromParameter(cfg->weights);	
tensor_t *M = cfg->weightMask;	
rigLUpdateMask(W, G, M, numToSwap);	
}	

11.9 -- rigLModelStep: Full Model Update

void rigLModelStep(layer_t **model, size_t modelSize,	
rigLModelConfig_t *cfg) {	
cfg->globalStep++;	<-- count every gradient step
/* Check if mask is frozen (all cycles done) */	
if (cfg->cyclesDone >= cfg->numCycles) return;	<-- mask frozen
/* Check if it is time to update (every updateFreq steps) */	
if (cfg->globalStep % cfg->updateFreq != 0) return;	<-- not yet
/* Update each sparse layer */	
for (size_t i=0; i<modelSize; i++) {	
layer_t *L = model[i];	
tensor_t *mask = getLayerWeightMask(L);	<-- NULL for dense layers
if (mask == NULL) continue;	<-- skip dense

size_t nnz = rigLCountActive(mask);	<-- active weights
size_t K = (size_t)(cfg->turnoverFrac * (float)nnz);	<-- weights to swap
if (K == 0) K = 1;	<-- always swap at least 1
switch (L->type) {	
case LINEAR:	
rigLLinearStep(L->config->linear, K); break;	
case CONV1D:	
rigLConv1dStep(L->config->conv1d, K); break;	
default: break;	<-- other layers skipped
}	
}	
cfg->cyclesDone++;	<-- one cycle completed
}	

11.10 -- rigLReMaskModel: Post-Optimizer Re-masking

void rigLReMaskModel(layer_t **model, size_t modelSize) {	
for (size_t i=0; i<modelSize; i++) {	
layer_t *L = model[i];	
tensor_t *mask = getLayerWeightMask(L);	
if (mask == NULL) continue;	<-- dense layer
tensor_t *W;	
switch (L->type) {	
case LINEAR:	
W =	
getParamFromParameter(L->config->linear->weights);	
break;	
case CONV1D:	
W =	
getParamFromParameter(L->config->conv1d->weights);	
break;	
default: continue;	
}	
rigLReMask(W, mask);	<-- zero any drift
}	
}	

CONCEPT: When to Call rigLReMaskModel

rigLReMaskModel must be called AFTER the optimizer step and BEFORE zero-grad:

Timeline per gradient step:

1. calculateGradsSequential() -> grads computed for all weights

2. `rigLModelStep()` -> prune+grow using grad (if update time)
3. `optimizer.step()` -> SGD updates all weights (including pruned!)
4. `rigLReMaskModel()` -> zero out drift in pruned weights <-- HERE
5. `optimizer.zero()` -> clear all gradients

If step 4 is skipped: pruned weights get nonzero values from SGD.

Next forward pass: those weights contribute to output -- sparsity violated.

After many steps: model is no longer sparse at all.

Chapter 12 -- End-to-End Integration Walkthrough

This chapter shows exactly how to wire RigL into an ODT training session for the LibriSpeech speaker-ID task on STM32 Nucleo-F746ZG.

12.1 -- Setup: Allocate Buffers

/* 1. Allocate mask buffers for each sparse layer. */	
/* Weight shape for Linear(256,128): [128, 256] = 32768 weights */	
/* Mask size: ceil(32768 / 8) = 4096 bytes. */	
static uint8_t linearMaskData[4096];	<-- stack or static buffer
static tensor_t linearMaskTensor;	
static rigLSparsityConfig_t linearSparsityCfg;	
/* 2. Init mask tensor (BOOL quantization, same shape as weights) */	
initBoolTensor(&linearMaskTensor, linearMaskData, linearWeights->shape);	<-- same shape as weight tensor
/* 3. Init RigL sparsity config */	
rigLInitSparsityConfig(&linearSparsityCfg, &linearMaskTensor,	<-- mask buffer
0.90f,	<-- target: 90% sparse
0.20f,	<-- swap 20% of active weights per cycle
100,	<-- update every 100 gradient steps
10);	<-- 10 prune+grow cycles total
/* 4. Wire mask into layer config */	
linearLayerCfg.weightMask = &linearMaskTensor;	
linearLayerCfg.sparsityConfig = &linearSparsityCfg;	
/* 5. Initialize mask from current weights (initial pruning) */	
tensor_t *W = getParamFromParameter(linearLayerCfg.weights);	
rigLInitMaskFromWeights(&linearMaskTensor, W, 0.90f);	<-- prune 90%

12.2 -- Setup: RigL Model Config

static rigLModelConfig_t rigLCfg;	
rigLInitModelConfig(&rigLCfg,	
100,	<-- T = 100 steps between updates
10,	<-- C = 10 total cycles
0.20f);	<-- Delta = 20% turnover per cycle

12.3 -- Training Loop with RigL

for (size_t epoch=0; epoch<numEpochs; epoch++) {	
for (size_t bi=0; bi<numBatches; bi++) {	
/* Step 1: forward + backward */	
trainingStats_t *stats = calculateGradsSequential(	
model, modelSize, lossConfig,	
REDUCTION_MEAN, batch->item, batch->label);	
/* Step 2: RigL mask update (uses grads -- BEFORE	
zero) */	
rigLModelStep(model, modelSize, &rigLCfg);	
/* Step 3: SGD step (updates all weights) */	
optimizerFunctions[optimizer->type].step(optimizer	
);	
/* Step 4: re-mask (zero drift in pruned weights)	
*/	
rigLReMaskModel(model, modelSize);	
/* Step 5: zero gradients (must be last) */	
optimizerFunctions[optimizer->type].zero(optimizer	
);	
freeTrainingStats(stats);	
}	
}	

CONCEPT: Critical Ordering: backward -> RigL -> step -> re-mask -> zero-grad

1. backward(): Computes grad for ALL weights (active and pruned).

2. rigLStep(): Uses grad[inactive] to find which to GROW.

Uses |w[active]| to find which to PRUNE.

Updates mask M. Newly grown: M[i]=1, w[i]=0.

Newly pruned: M[i]=0, w[i]=0.

3. step(): SGD updates w[i] for all i.

4. re-mask(): w[i] = 0 for all i where M[i]==0. Clears drift.

5. zero-grad(): Clears all gradients. Must be LAST.

If zero-grad() is called before rigLStep(): grow step has no signal.

If re-mask() is skipped: pruned weights accumulate SGD updates.

12.4 -- Sparsity Monitoring

float sp = rigLMeasureSparsity(&linearMaskTensor);	
printf("Linear sparsity: %.1f%%\n", sp * 100.f);	
printf("Active: %zu / %zu\n",	
rigLCountActive(&linearMaskTensor),	

<code>calcNumberOfElementsByTensor(&linearMaskTensor));</code>	
<code>printf("RigL cycle %zu / %zu\n",</code>	
<code>rigLCfg.cyclesDone, rigLCfg.numCycles);</code>	

CONCEPT: Expected Training Behavior for Speaker ID

LibriSpeech 100h speaker ID with Linear(256,128) sparse at 90%:

Step 0: `rigLInitMaskFromWeights()` prunes 90% of 32768 weights.

Only 3277 active weights.

Forward pass: 90% fewer MACs in Linear layer.

Steps 1-99: Standard sparse forward+backward. Mask unchanged.

Step 100: First prune+grow. $K = \text{floor}(0.20 \times 3277) = 655$ weights swapped.

Model may lose ~1-2% accuracy transiently.

Steps 100-1000: 10 cycles. Active set iteratively improves.

Step 1001+: Mask frozen. Pure sparse inference and training.

Memory saved: $32768 \times 4\text{B} = 128\text{KB}$ (weights) $\rightarrow 3277 \times 4\text{B} = 13\text{KB}$ effective.

Plus mask: 4096B. Net: $128\text{KB} \rightarrow 17\text{KB} = 87\%$ memory reduction.

Compute saved: ~90% fewer MACs in Linear forward pass.

Glossary

active weight

A weight where $\text{mask}[i]=1$. Participates in forward and backward.

cyclesDone

Number of completed prune+grow cycles. When $\geq \text{numCycles}$, mask frozen.

drift

Non-zero values in pruned ($\text{mask}=0$) positions after optimizer step. Cleared by `rigLReMask()`.

findAbsKthSmallest

Selection-sort $O(k*N)$, $O(1)$ space. Finds k-th smallest absolute value.

frozen mask

After numCycles , mask never modified. Remaining training uses fixed sparse topology.

getLayerWeightMask

Returns BOOL mask tensor for a layer, or NULL if dense.

globalStep

Counter in `rigLModelConfig_t`. Incremented every gradient step.

GROW step

Activates K inactive weights with largest $|\text{grad}|$. Sets $\text{mask}[i]=1$, $w[i]=0$.

matmulWithMask

Sparse matmul: skips MAC when $\text{mask}[\text{bldx}]=0$. New function in `Matmul.c`.

numCycles

Total prune+grow cycles C. After numCycles , mask frozen.

numToSwap

$K = \text{floor}(\text{turnoverFrac} * \text{nnz}(\text{mask}))$. Weights pruned and grown per cycle.

PRUNE step

Deactivates K active weights with smallest $|w|$. Sets $\text{mask}[i]=0$, $w[i]=0$.

re-masking

After optimizer step: zero $w[i]$ where $\text{mask}[i]=0$ to remove drift.

RigL

Rigging the Lottery (Evci et al. 2020): dynamic sparse training at constant sparsity.

rigLApplyMask

Zeros $w[i]$ for all i where $\text{mask}[i]=0$.

rigLInitMaskFromWeights

Prunes to targetSparsity by zeroing smallest $|w|$ weights.

rigLModelConfig_t

Model-level RigL schedule: T, C, Delta, globalStep, cyclesDone.

rigLModelStep

Checks schedule; if time, calls per-layer rigLLinearStep/rigLConv1dStep.

rigLReMaskModel

Calls rigLReMask for all sparse layers after optimizer step.

rigLSparsityConfig_t

Per-layer config: mask, targetSparsity, turnoverFrac, updateFreq, numCycles.

rigLUpdateMask

Core: prune K active (smallest $|w|$), grow K inactive (largest $|\text{grad}|$).

sparsityConfig

Empty struct in Tensor.h -- replaced by rigLSparsityConfig_t for RigL.

targetSparsity

Fraction of zero weights. 0.90 = 90% pruned.

tensorBoolGet/Set

Bit-level access to BOOL tensor mask.

turnoverFrac

Fraction of active weights swapped per cycle (Delta). Typical: 0.20.

updateFreq

T: gradient steps between mask updates. Typical: 100.

weightMask

BOOL tensor field added to linearConfig_t, conv1dConfig_t. NULL = dense.